IUM Projekt – Etap 2

Zespół : Krzysztof Barałkiewicz, Tomasz Nejman

Opis problemu

Celem było zrobienie serwisu, który ułatwi życie użytkownikom dodającym oferty wynajmu. Zamiast ręcznie klikać w formularzu liczbę łóżek czy typ pokoju, użytkownik wpisuje tylko opis tekstowy, a nasz system sam wypełnia resztę pól.

Zastosowane modele i inżynieria cech

Implementacja znajduje się w pliku model2.py. Stworzyliśmy interfejs PredictionModel i dwie konkretne klasy, implementujące modele: Baseline i Advanced (to ten z ML), aby łatwo przełączać się między strategiami.

Model Bazowy (Baseline) Służy jako punkt odniesienia. Nie analizuje treści opisu, opiera się wyłącznie na statystykach ze zbioru treningowego:

- Dla klasyfikacji (room_type, etc.): Zwraca dominantę (modę) – najczęstszą wartość.
- Dla regresji (bedrooms, beds): Zwraca medianę, aby zminimalizować wpływ wartości odstających.
- Udogodnienia: Wykrywane są za pomocą wyrażeń regularnych (Regex) na podstawie słów kluczowych (np. "wifi" -> Wifi).

Model Zaawansowany (Advanced) Wykorzystuje bibliotekę Scikit-Learn i potoki przetwarzania (Pipeline). Proces wygląda następująco:

- Preprocessing (TextCleaner):
  - Czyszczenie tekstu: usuwanie tagów HTML, znaków specjalnych i zamiana na małe litery.
  - Mapowanie liczebników: Zamiana słów na cyfry (np. "two bedrooms" -> "2 bedrooms", "studio" -> "0").
- Reprezentacja tekstu (TF-IDF):
  - Użyliśmy TfidfVectorizer z ograniczeniem do 5000 cech.
  - Zastosowaliśmy n-gramy (1, 2), aby wyłapać pary słów (np. "high ceilings").
  - Usunęliśmy angielskie stop words.
- Algorytmy uczące:
  - Dla zmiennych kategorycznych: Regresja Logistyczna (LogisticRegression).
  - Dla zmiennych liczbowych: Regresja Ridge z parametrem alpha=1.

W obu modelach ekstrakcja udogodnień (amenities) działa tak samo (Regex), ponieważ jest to metoda deterministyczna i wystarczająco skuteczna.

Trenowanie i ewaluacja

Najpierw przetestowaliśmy modele "na sucho" na zbiorze danych listings.csv. Podzieliliśmy dane na treningowe i testowe (stosunek 4:1), żeby zobaczyć, o ile ML jest lepszy od strzelania medianą.
Logi z trenowania i testów (z pliku model2.py):

```
--- Evaluation on Test Set ---
           Target    Metric  Base model score  Advanced model score  Improvement  Status
0       room_type  Accuracy            0.7485                0.8421       0.0936  SUKCES
1   property_type  Accuracy            0.5923                0.6336       0.0413  SUKCES
2  bathrooms_text  Accuracy            0.6127                0.6387       0.0260  SUKCES
3        bedrooms       MAE            0.4092                0.3385       0.0708  SUKCES
4            beds       MAE            0.6082                0.5149       0.0933  SUKCES
5    accommodates       MAE            1.0028                0.8977       0.1051  SUKCES
6       amenities   Jaccard            0.0431                0.0431       0.0000  SUKCES
```

Notka: W przypadku “amenities” używany jest regex, a do ewaluacji Jaccard. Jak wiadomo regex jest deterministyczny dla tych samych danych, stąd ten sam wynik.

Mikroserwis i API

Wystawiliśmy modele jako API używając FastAPI z swaggerem (/docs), który umożliwia nam lepszą dokumentację. Mamy trzy główne endpointy:

1. /app/predict/baseline – wymusza użycie modelu bazowego.
2. /app/predict/advanced – wymusza użycie modelu ML.
3. /app/predict/ab_test – Najważniejszy. Losowo (50/50) wybiera model, żebyśmy mogli przeprowadzić eksperyment A/B.

Dodatkowo jest endpoint /app/feedback, gdzie użytkownik wysyła poprawki, informację zwrotną jakie finalnie wartości wykorzystał. Jest to potrzebne do testu A/B.

Eksperyment A/B

Eksperyment polega na użyciu endpointu ab_test, który losowo wydaje model i zapisuje z niego odpowiedzi w logach. Następnie użytkownik przez podanie id predykcji oraz swoich danych które naprawdę użył odsyła je na serwer. Po zebranych logach możemy uruchomić program analize_ab, który porówna logi i obliczy nam wyniki testu.
Przykładowe wnioski z testu ab:

```
Total merged records (Prediction + Feedback): 200

==================================================
 A/B TEST RESULTS SUMMARY
==================================================
Model                baseline  advanced
Count                 95.0000  105.0000
Acc_room_type          0.7579    0.8000
Acc_property_type      0.4526    0.5429
```

```
Acc_bathrooms_text    0.6105    0.7619
MAE_bedrooms          0.5053    0.4286
MAE_beds              0.5368    0.2762
MAE_accommodates      1.0211    0.5238
Jaccard_amenities     0.4947    0.5975
==================================================
```

Notka: W przypadku “amenities” używany jest regex, a do ewaluacji Jaccard. W przypadku testu A/B dane nie są jednakowe dla obu modeli stąd różne wyniki Jaccarda, pomimo deterministycznego charakteru regexa.

Wnioski

1. Model Advanced działa lepiej, widać to po mniejszym błędzie (MAE) i wyższym Accuracy w porównaniu do modelu bazowego.
2. Analiza tekstu ma sens, proste statystyki (baseline) słabo sobie radzą z nietypowymi ofertami, a model ML wyłapuje kontekst.
3. Regex do “amenities” jest dostatecznie skuteczny, ale tak naprawdę zależy od tego jak dokładnie zdefiniujemy słowa kluczowe.
4. System jest gotowy do wpięcia w aplikację. Skraca czas wypełniania formularza, bo użytkownik musi tylko poprawić błędy, a nie wpisywać wszystkiego od zera.